

Greeks-optimized Portfolios under Stochastic Volatility Models

Léa Malergue & Amory Goulet
Supervisor : Mario Hernandez Tinoco

May 2025

Abstract

This thesis aims to show how the choice of a stochastic volatility model impacts the risk-adjusted performance of Greeks-optimized option portfolios. Focusing on the Heston and SABR stochastic volatility models, we build a portfolio of European options using simulated prices derived from the Black-Scholes model, apply optimization constraints on Delta, Gamma, and Vega, and compare outcomes across different market conditions.

Keywords: Portfolio Optimization, Greeks, Heston, SABR, Stochastic Volatility

Contents

1	Introduction	4
2	Theoretical Background	5
2.1	Fundamentals of Options	5
2.1.1	Payoff Functions	5
2.1.2	Moneyness	6
2.1.3	Option Pricing as a Function of Parameters	6
2.2	The Greeks: Sensitivities of Option Prices	6
2.2.1	Key Greeks	6
2.2.2	Portfolio Greeks and Optimization	7
2.3	Option Pricing: The Black-Scholes-Merton (BSM) Model	8
2.3.1	Model Assumptions	8
2.3.2	Geometric Brownian Motion	8
2.3.3	Constructing a Riskless Portfolio	8
2.3.4	The Black-Scholes Partial Differential Equation	9
2.3.5	Closed-Form Solutions for European Options	9
2.4	Option Pricing: The Heston Stochastic Volatility Model	10
2.4.1	Model Assumptions	10
2.4.2	Risk-Neutral Valuation Framework	11
2.4.3	The Heston Partial Differential Equation	11
2.4.4	Heston Closed-Form Solution for European Options	13
2.4.5	Option Greeks under the Heston Model	13
2.5	Option Pricing: The SABR Stochastic Volatility Model	14
2.5.1	Model Assumptions	14
2.5.2	Approximate Pricing Formula	15
2.5.3	Option Greeks under the SABR Model	15
3	Methodology and Implementation	17
3.1	Selection of the Underlying Asset and the Study Period	17
3.2	Option Data Construction	18
3.3	Portfolio Objectives and Optimization Constraints	20
3.4	Implementation and Results of the Optimization	21
3.4.1	Optimization under the SABR Model	21
3.4.2	Optimization under the Heston Model	23
3.5	Backtesting and Performance evaluation	24
3.5.1	Backtesting Periods	24
3.5.2	Performance Evaluation Metrics	25
4	Results and Discussion	26
4.1	Empirical Findings	26
4.2	Critical Reflections	27
5	Conclusion	28

List of Figures

1	European Option Payoff Diagrams	5
2	WTI Crude Oil Futures Prices (2015-2025)	17
3	Sample of WTI Crude Oil Prices in our dataset	18
4	Crude Oil Options generated using Black-Scholes (2020-03-02)	19
5	Crude Oil Options generated using Black-Scholes (2021-01-04)	20
6	Crude Oil Options generated using Black-Scholes (2021-07-01)	20
7	Optimized portfolio using the SABR model (2020-03-02)	22
8	Optimized portfolio using the SABR model (2021-01-04)	22
9	Optimized portfolio using the SABR model (2021-07-01)	23
10	Optimized portfolio using the Heston model (2020-03-0Z)	23
11	Optimized portfolio using the Heston model (2021-01-04)	24
12	Optimized portfolio using the Heston model (2021-07-01)	24
13	Profit and Loss: Heston and SABR	26
14	Profit and Loss per unit of Vega: Heston and SABR	26

List of Tables

1	Moneyness conditions for call and put options	6
2	Main Greeks and their interpretations	7
3	Model Comparison Across Market Conditions	27

1 Introduction

Options play a crucial role in financial markets. Their attractiveness lies in their various uses: they allow for specific risk exposures and non-linear payoffs that are otherwise unattainable with simple assets. Yet, fully understanding their inner mechanisms requires rigorous quantitative modeling. One of the most critical challenges for investors in using options is choosing a suitable framework to compute option prices and Greeks to maximize option portfolio performance.

The Black-Scholes-Merton (BSM) model (1973) is the foundational model which option pricing has heavily relied on historically. In theory, it is convenient because the model assumes constant volatility and yields closed-form solutions for European options. However, in practice, the BSM model fails to capture important market phenomena such as the stochastic nature of volatility, which brings us to more advanced pricing models, such as the Heston model and the SABR model.

Both models introduce stochastic volatility, but feature different assumptions: the Heston model assumes mean-reverting variance with a square-root process, while the SABR model assumes a lognormal process without mean reversion. These differences ultimately impact option pricing and Greeks computation. There are several studies with the purpose of showing how Greeks differ under these models. However, not much has been done in evaluating how model choice, specifically stochastic models, impacts portfolio construction and optimization, leading us to the central question of our research: *How much does the choice of stochastic volatility model influence the risk-adjusted performance of an option portfolio, when Greeks are used as a base for optimization?*

2 Theoretical Background

2.1 Fundamentals of Options

Options are financial derivatives. They are linked to an *underlying asset* and give the right, but not the obligation, to buy or sell an underlying asset at a fixed price (*strike*) on or before a specific date. There are two basic types:

- **Call Option:** Right to buy an asset at a specific price on or before a specific date.
- **Put Option:** Right to sell an asset at the strike price on or before a specific date.

This study focuses exclusively on **European options**, which can only be exercised at maturity T . This constraint simplifies pricing and Greeks calculations notably by allowing closed-form solutions.

2.1.1 Payoff Functions

The payoffs at maturity T for European options are:

$$\begin{aligned} \text{Long European Call:} & \quad \max(S_T - K, 0) \\ \text{Short European Call:} & \quad -\max(S_T - K, 0) \\ \text{Long European Put:} & \quad \max(K - S_T, 0) \\ \text{Short European Put:} & \quad -\max(K - S_T, 0) \end{aligned}$$

where S_T is the underlying price at maturity and K the strike price.

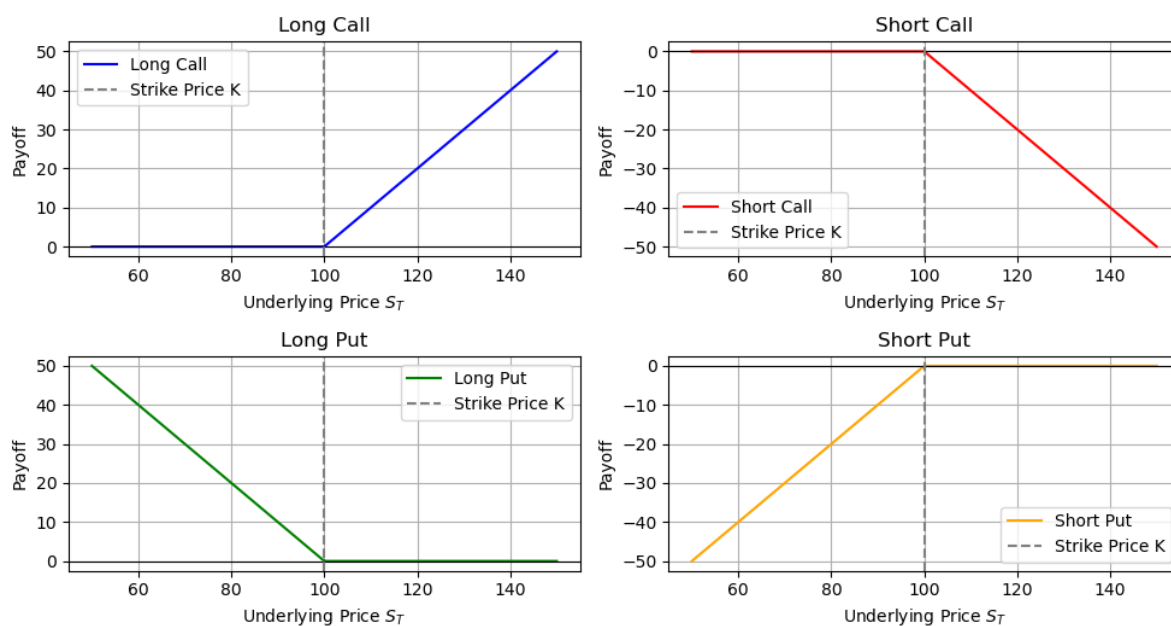


Figure 1: European Option Payoff Diagrams

The payoffs depend on the position taken: long or short. The buyer of an option takes a long position and the seller takes a short position. While the holder (buyer) has the right to buy or sell, the writer (seller) has to respect the buyer's decision to exercise or not.

2.1.2 Moneyness

The moneyness of a given option describes the relative position of the current price of its underlying asset (S) with respect to its strike price (K). Moneyness influences option pricing. For instance, out-of-the-money options tend to have lower premiums.

Moneyness	Call Option	Put Option
In the Money (ITM)	$S > K$	$S < K$
At the Money (ATM)	$S \approx K$	$S \approx K$
Out of the Money (OTM)	$S < K$	$S > K$

Table 1: Moneyness conditions for call and put options

2.1.3 Option Pricing as a Function of Parameters

The price of an option is a function $V(S, K, \sigma, r, t)$ that depends on several inputs:

- Current underlying price S
- Strike price K
- Time to maturity $T - t$
- Volatility σ
- Risk-free interest rate r
- Dividend yield q (*not considered in this study*)

To simplify the modeling process throughout this study, we will assume that the underlying asset does not pay dividends during the life of the option.

2.2 The Greeks: Sensitivities of Option Prices

Option Greeks are partial derivatives of the option pricing function with respect to key input variables. They show how option prices respond to certain changes in market conditions.

2.2.1 Key Greeks

The most important Greeks are Delta, Gamma, Vega, Theta, and Rho. In this thesis, we will focus on Delta, Gamma, and Vega, as they are most relevant for constructing and optimizing option portfolios. They can be described as follows:

Greek	Formula	Interpretation
Delta (Δ)	$\frac{\partial V}{\partial S}$	Sensitivity of the option price to changes in the underlying asset price. Ranges from 0 to 1 for calls, and -1 to 0 for puts.
Gamma (Γ)	$\frac{\partial^2 V}{\partial S^2}$	Sensitivity of Delta to changes in the underlying asset price; reflects the convexity of the option value.
Vega (ν)	$\frac{\partial V}{\partial \sigma}$	Sensitivity of the option price to changes in the volatility of the underlying asset.
Theta (Θ)	$\frac{\partial V}{\partial t}$	Sensitivity of the option price to the passage of time (time decay).
Rho (ρ)	$\frac{\partial V}{\partial r}$	Sensitivity of the option price to changes in the risk-free interest rate.

Table 2: Main Greeks and their interpretations

2.2.2 Portfolio Greeks and Optimization

In traditional portfolio theory, optimization focuses on expected return and variance. But risk is more complex in option portfolios, and managing the exposures to the underlying price (Delta), convexity (Gamma) and volatility (Vega) is essential. For a portfolio consisting of n option positions with weights w_i , the portfolio's overall exposure to a given Greek is the weighted sum of individual option exposures:

$$G^P = \sum_{i=1}^n w_i G_i$$

where G^P is the portfolio Greek (such as Delta, Gamma, or Vega) and G_i is the Greek of the i -th option. This allows us to build a portfolio by selecting the weights w_i to satisfy different preferences, such as:

- Delta-neutral portfolios ($\Delta^P = 0$) to hedge against small moves in the underlying
- Gamma-limited or Vega-limited portfolios, which limit overall exposure to convexity (Gamma) or volatility (Vega)
- Directional strategies with portfolios maximizing a particular Greek, such as Vega

The optimization problem is typically formulated as:

Maximize or minimize: $f(\mathbf{w})$

Subject to:

$$\sum_{i=1}^n w_i = 1$$

$$-1 \leq w_i \leq 1$$

Risk constraints (e.g., $|\Gamma^P| \leq \text{threshold}$)

where $f(\mathbf{w})$ is the objective function (e.g. maximize Vega, etc.).

2.3 Option Pricing: The Black-Scholes-Merton (BSM) Model

The Black-Scholes-Merton (BSM) model, introduced in 1973 by Fischer Black, Myron Scholes, and Robert Merton, allows for pricing European options by modeling the underlying asset price as a geometric Brownian motion under specific market conditions.

2.3.1 Model Assumptions

The BSM model relies on several idealized assumptions that simplify financial market dynamics and enable closed-form solutions for European options:

- Underlying asset price follows a Geometric Brownian Motion with constant drift μ and volatility σ
- Constant and known risk-free interest rate, independent of maturity
- No dividends paid during the option's life
- Markets are frictionless (no transaction costs, bid-ask spreads, taxes)
- Short selling allowed with full use of proceeds
- Continuous trading and no arbitrage opportunities

2.3.2 Geometric Brownian Motion

The Black-Scholes-Merton model assumes that the underlying asset price S_t follows a *Geometric Brownian Motion* (GBM), a continuous-time stochastic process that captures both the predictable trend and the random fluctuations in asset prices, and that is defined by the stochastic differential equation:

$$dS_t = \mu S_t dt + \sigma S_t dW_t \quad (1)$$

where:

- μ is the constant *drift* term, representing the expected return of the asset
- σ is the constant *volatility*, representing the standard deviation of returns
- W_t is a *standard Brownian motion* (or Wiener process), modeling the random fluctuations

2.3.3 Constructing a Riskless Portfolio

It is possible to construct a *locally riskless* portfolio by combining a position in the option with a position in the underlying asset that eliminates the uncertainty caused by the random fluctuations of the underlying asset price. Let $f(S_t, t)$ denote the price of the option at time t . Consider the portfolio:

$$\Pi = -f + \Delta S_t \quad (2)$$

where $\Delta = \frac{\partial f}{\partial S}$ is the option's sensitivity to the underlying price (Delta). The first term is a short position in the option, and the second term is a long position in the underlying asset. The infinitesimal change in the portfolio value over a small time interval dt is:

$$d\Pi = -df + \Delta dS_t \quad (3)$$

Applying Itô's lemma to $f(S_t, t)$, which is assumed to be once continuously differentiable in t and twice in S we have:

$$df = \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial S} dS_t + \frac{1}{2} \frac{\partial^2 f}{\partial S^2} \sigma^2 S_t^2 dt \quad (4)$$

Substituting (4) into (3), and recalling that the underlying price dynamics are given by (1) :

$$dS_t = \mu S_t dt + \sigma S_t dW_t,$$

We observe that the stochastic term dW_t cancels out exactly:

$$d\Pi = - \left(\frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial S} dS_t + \frac{1}{2} \frac{\partial^2 f}{\partial S^2} \sigma^2 S_t^2 dt \right) + \Delta dS_t = - \frac{\partial f}{\partial t} dt - \frac{1}{2} \frac{\partial^2 f}{\partial S^2} \sigma^2 S_t^2 dt + \left(\Delta - \frac{\partial f}{\partial S} \right) dS_t$$

Since $\Delta = \frac{\partial f}{\partial S}$, the term multiplying dS_t vanishes, removing all randomness from the portfolio change:

$$d\Pi = - \frac{\partial f}{\partial t} dt - \frac{1}{2} \frac{\partial^2 f}{\partial S^2} \sigma^2 S_t^2 dt \quad (5)$$

This means the portfolio's value evolves deterministically over the interval dt : it is *locally riskless*. Therefore, by continuously adjusting the number of underlying shares held, an investor can hedge away the random fluctuations in the option price.

2.3.4 The Black-Scholes Partial Differential Equation

The principle of no arbitrage dictates that the return on the riskless portfolio Π must equal the risk-free interest rate r . Therefore,

$$d\Pi = r\Pi dt. \quad (6)$$

Substituting (2) & (5) into (6), we arrive at the Black-Scholes partial differential equation:

$$\frac{\partial f}{\partial t} + rS_t \frac{\partial f}{\partial S} + \frac{1}{2} \sigma^2 S_t^2 \frac{\partial^2 f}{\partial S^2} = rf. \quad (7)$$

The Black-Scholes PDE characterizes how the option price $f(S_t, t)$ changes over time with respect to changes in the underlying asset price. The terms represent:

- $\frac{\partial f}{\partial t}$: the sensitivity of the option price to the passage of time (**Theta** Θ)
- $rS_t \frac{\partial f}{\partial S}$: the expected growth in option price due to the underlying asset's price drift at the risk-free rate
- $\frac{1}{2} \sigma^2 S_t^2 \frac{\partial^2 f}{\partial S^2}$: accounts for how the option's convexity (**Gamma** Γ) and underlying volatility together influence its price

2.3.5 Closed-Form Solutions for European Options

With conditions given earlier by the option payoff at maturity T :

$$\begin{aligned} f(S_T, T) &= \max(S_T - K, 0) \quad \text{for a call option} \\ f(S_T, T) &= \max(K - S_T, 0) \quad \text{for a put option} \end{aligned}$$

The solution to the PDE leads to the Black-Scholes formulas:

$$C = S_0 N(d_1) - K e^{-rT} N(d_2), \quad P = K e^{-rT} N(-d_2) - S_0 N(-d_1) \quad (8)$$

where:

- C and P are the prices of the European call and put options respectively
- S_0 is the current price of the underlying asset
- $N(\cdot)$ is the cumulative distribution function of the standard normal distribution

The terms d_1 and d_2 are given by:

$$d_1 = \frac{\ln\left(\frac{S_0}{K}\right) + \left(r + \frac{\sigma^2}{2}\right)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T},$$

and represent standardized variables related to the probability that the option finishes in-the-money under the risk-neutral measure.

2.4 Option Pricing: The Heston Stochastic Volatility Model

The Heston model, introduced by Steven Heston in 1993, extends classical option pricing by incorporating **stochastic volatility**. This addresses empirical observations that volatility varies over time and is itself random.

2.4.1 Model Assumptions

Unlike the constant volatility assumption in Black-Scholes-Merton, the Heston model assumes that both the underlying asset price S_t and variance v_t follow stochastic processes:

$$\begin{cases} dS_t = \mu S_t dt + \sqrt{v_t} S_t dW_t^S \\ dv_t = \kappa(\theta - v_t)dt + \sigma_v \sqrt{v_t} dW_t^v \end{cases}$$

where:

- μ is the drift rate of the asset price, representing its expected return (like in BSM)
- $\kappa > 0$ is the rate of mean reversion of variance towards the long-run mean θ
- $\theta > 0$ is the long-term variance mean (as t approaches infinity, v_t tends towards θ)
- $\sigma_v > 0$ is the volatility of variance ("vol of vol"): higher values increase the randomness of variance, this raises the kurtosis of the return distribution (fatter tails)
- W_t^S and W_t^v are correlated Brownian motions with $\rho = \text{Corr}(dW_t^S, dW_t^v)$

The process for v_t is a **Cox-Ingersoll-Ross (CIR)** square-root process, which ensures non-negative variance. To ensure that variance remains strictly positive (does not hit zero), the **Feller condition** must be satisfied. This condition ensures a balance between how strongly the variance is pulled back to its long-term average ($\kappa\theta$) and how erratic it is due to randomness (σ_v^2). This avoids the variance from being zero :

$$2\kappa\theta \geq \sigma_v^2$$

2.4.2 Risk-Neutral Valuation Framework

To price options, we do not work under the real-world probability measure $\mathbb{P}$, where asset returns reflect risk premia. Indeed, that would require knowing investors' utility functions (preferences, risk aversion) to get the market price of risk, which is impossible in practice.

Instead, we adopt a **risk-neutral measure** $\mathbb{Q}$, where investors are indifferent to risk (hence *risk-neutral*), in which all assets are assumed to grow, on average, at the risk-free rate r . This allows for option pricing by discounting their expected future payoffs at the rate r . The principle of no-arbitrage ensures that this risk-neutral measure is unique.

Under $\mathbb{Q}$, the dynamics are as follows:

$$\begin{cases} dS_t = rS_t dt + \sqrt{v_t} S_t d\widetilde{W}_t^S \\ dv_t = \kappa^*(\theta^* - v_t)dt + \sigma_v \sqrt{v_t} d\widetilde{W}_t^v \end{cases} \quad (9)$$

A few adjustments are made to ensure that the model matches observed market prices of options. The drift μ of the Brownian motions (now $\widetilde{W}_t^S$ and $\widetilde{W}_t^v$) is adjusted to the risk-free rate r . Then, the shift from the physical parameters κ, θ to κ^*, θ^* accounts for the fact that investors demand compensation for the volatility risk. In the real world, market prices already include compensation for volatility risk. Here, we need to adjust the parameters so that the risk-neutral valuation formulas produce market prices:

$$\kappa^* = \kappa + \lambda, \quad \theta^* = \frac{\kappa\theta}{\kappa^*},$$

and λ is the *market price of volatility risk*.

2.4.3 The Heston Partial Differential Equation

Building on the derivation of the BSM Partial Differential Equation, we will now extend the approach by allowing the volatility to be stochastic.

First, we will consider the price of a European option $V(S, v, t)$ at time t as a function of the underlying price S , the instantaneous variance v , and time t , under the risk-neutral measure $\mathbb{Q}$. Other inputs like the strike price K , the risk-free rate r or the model parameters $\kappa, \theta, \sigma_v, \rho$ are all treated as constants during pricing.

Second, we will consider the following portfolio:

$$\Pi = -V + \Delta S_t + \Gamma v_t \quad (10)$$

This riskless portfolio Π consists of a short position in the option, a long position in Δ shares of the underlying asset S_t to hedge exposure to price risk dW_t^S and a long position in Γ units in a variance-sensitive instrument, such as a derivative dependent on v_t to hedge exposure to the volatility risk dW_t^v . The infinitesimal change in the portfolio value over a small time interval dt is:

$$d\Pi = -dV + \Delta dS_t + \Gamma dv_t \quad (11)$$

We assume that $V(S, v, t) \in C^{1,2,2}$, meaning the option price function is once continuously differentiable in time t , and twice continuously differentiable in space, both S and v . We can therefore apply Itô's Lemma to the function $V(S, v, t)$, obtaining:

$$dV = \frac{\partial V}{\partial t} dt + \frac{\partial V}{\partial S} dS_t + \frac{\partial V}{\partial v} dv_t + \frac{1}{2} \frac{\partial^2 V}{\partial S^2} (dS_t)^2 + \frac{1}{2} \frac{\partial^2 V}{\partial v^2} (dv_t)^2 + \frac{\partial^2 V}{\partial S \partial v} dS_t dv_t \quad (12)$$

To compute the second-order terms, we use properties of stochastic calculus:

$$(dW_t^S)^2 = dt, \quad (dW_t^v)^2 = dt, \quad dW_t^S \cdot dW_t^v = \rho dt$$

Recalling the risk-neutral stochastic processes (9), we derive the squares and cross terms:

$$(dS_t)^2 = v_t S_t^2 dt, \quad (dv_t)^2 = \sigma_v^2 v_t dt, \quad dS_t dv_t = \rho \sigma_v v_t S_t dt$$

Substituting this into the expansion of dV (12), we obtain:

$$dV = \frac{\partial V}{\partial t} dt + \frac{\partial V}{\partial S} dS_t + \frac{\partial V}{\partial v} dv_t + \frac{1}{2} \frac{\partial^2 V}{\partial S^2} v_t S_t^2 dt + \frac{1}{2} \frac{\partial^2 V}{\partial v^2} \sigma_v^2 v_t dt + \frac{\partial^2 V}{\partial S \partial v} \rho \sigma_v v_t S_t dt$$

Substituting this into equation (11) we obtain:

$$d\Pi = - \left[\frac{\partial V}{\partial t} dt + \frac{\partial V}{\partial S} dS_t + \frac{\partial V}{\partial v} dv_t + \frac{1}{2} \frac{\partial^2 V}{\partial S^2} v_t S_t^2 dt + \frac{1}{2} \frac{\partial^2 V}{\partial v^2} \sigma_v^2 v_t dt + \frac{\partial^2 V}{\partial S \partial v} \rho \sigma_v v_t S_t dt \right] + \Delta dS_t + \Gamma dv_t$$

Grouping terms, we get:

$$d\Pi = \left(-\frac{\partial V}{\partial t} - \frac{1}{2} \frac{\partial^2 V}{\partial S^2} v_t S_t^2 - \frac{1}{2} \frac{\partial^2 V}{\partial v^2} \sigma_v^2 v_t - \frac{\partial^2 V}{\partial S \partial v} \rho \sigma_v v_t S_t \right) dt + \left(-\frac{\partial V}{\partial S} + \Delta \right) dS_t + \left(-\frac{\partial V}{\partial v} + \Gamma \right) dv_t$$

We have to choose Δ and Γ such that the dW_t^S and dW_t^v terms cancel out:

$$\Delta = \frac{\partial V}{\partial S}, \quad \Gamma = \frac{\partial V}{\partial v}$$

The randomness is eliminated, the portfolio therefore becomes riskless and deterministic:

$$d\Pi = \left(-\frac{\partial V}{\partial t} - \frac{1}{2} \frac{\partial^2 V}{\partial S^2} v_t S_t^2 - \frac{1}{2} \frac{\partial^2 V}{\partial v^2} \sigma_v^2 v_t - \frac{\partial^2 V}{\partial S \partial v} \rho \sigma_v v_t S_t \right) dt \quad (13)$$

Finally, the no-arbitrage condition requires:

$$d\Pi = r\Pi dt = r \left(-V + \frac{\partial V}{\partial S} S_t + \frac{\partial V}{\partial v} v_t \right) dt \quad (14)$$

Equating both expressions (13), (14) for $d\Pi$ gives the Heston partial differential equation:

$$\frac{\partial V}{\partial t} + r S_t \frac{\partial V}{\partial S} + \kappa^* (\theta^* - v_t) \frac{\partial V}{\partial v} + \frac{1}{2} v_t S_t^2 \frac{\partial^2 V}{\partial S^2} + \rho \sigma_v v_t S_t \frac{\partial^2 V}{\partial S \partial v} + \frac{1}{2} \sigma_v^2 v_t \frac{\partial^2 V}{\partial v^2} = rV$$

2.4.4 Heston Closed-Form Solution for European Options

While the Heston PDE does not have a closed-form solution in the traditional sense as in the Black-Scholes formula, Heston showed that it was possible to derive a semi-closed form solution for European prices. To solve this PDE for a European call with strike K and maturity T , Heston proposes a solution structure inspired by Black-Scholes:

$$C(S_t, v_t, t) = S_t P_1 - K e^{-r(T-t)} P_2,$$

where P_1 and P_2 are risk-neutral probabilities:

- P_1 : Probability that the option expires in the money ($S_T > K$), used to discount the stock component.
- P_2 : Probability that the option is exercised, used to discount the strike component.

The probabilities are retrieved via the inverse Fourier transform on the characteristic functions $\phi_j(u)$ of $\log S_T$ under the risk-neutral measure and are defined as:

$$P_j = \frac{1}{2} + \frac{1}{\pi} \int_0^\infty \operatorname{Re} \left[\frac{e^{-iu \log K} \phi_j(u)}{iu} \right] du, \quad \text{for } j = 1, 2$$

where $\phi_j(u)$ is the characteristic function of $\log S_T$. Specifically:

$$\phi_j(u) = \exp(C_j(u, \tau) + D_j(u, \tau)v + iu \log S), \quad \tau = T - t$$

The functions $C_j(u, \tau)$ and $D_j(u, \tau)$ are defined as:

$$D_j(u, \tau) = \frac{b_j - \rho\sigma_v iu + d_j}{\sigma_v^2} \left(\frac{1 - e^{d_j \tau}}{1 - g_j e^{d_j \tau}} \right)$$

$$C_j(u, \tau) = r iu \tau + \frac{\kappa \theta}{\sigma_v^2} \left[(b_j - \rho\sigma_v iu + d_j) \tau - 2 \log \left(\frac{1 - g_j e^{d_j \tau}}{1 - g_j} \right) \right]$$

with:

$$d_j = \sqrt{(\rho\sigma_v iu - b_j)^2 - \sigma_v^2(2ui + \delta_j u^2)}, \quad g_j = \frac{b_j - \rho\sigma_v iu + d_j}{b_j - \rho\sigma_v iu - d_j},$$

$$b_1 = \kappa + \lambda - \rho\sigma_v, \quad b_2 = \kappa + \lambda, \quad \delta_1 = 1, \quad \delta_2 = 0$$

2.4.5 Option Greeks under the Heston Model

The closed-form solution derived by Heston expresses the price of a European option as a function of integrals involving the characteristic function of log-returns. So while the Black-Scholes model provides closed-form formulas for the Greeks, the Heston model, with its stochastic volatility, require a more complex approach.

To compute Greeks under the Heston model, we apply **numerical differentiation**, specifically using **finite difference methods**. This approach approximates partial

derivatives of the option price with respect to input variables by evaluating the option price at slightly perturbed values.

Let $C(S, v, t)$ be the price of a European call option under the Heston model. We approximate the primary Greeks as follow:

$$\begin{aligned}\text{Delta } \Delta &\approx \frac{C(S + \epsilon, v, t) - C(S - \epsilon, v, t)}{2\epsilon} \\ \text{Gamma } \Gamma &\approx \frac{C(S + \epsilon, v, t) - 2C(S, v, t) + C(S - \epsilon, v, t)}{\epsilon^2} \\ \text{Vega } \nu &\approx \frac{C(S, v + \epsilon, t) - C(S, v - \epsilon, t)}{2\epsilon}\end{aligned}$$

for a small perturbation $\epsilon > 0$ in S (small enough to ensure an accurate approximation but not so small that numerical instability arises).

2.5 Option Pricing: The SABR Stochastic Volatility Model

The SABR model (Stochastic Alpha Beta Rho), introduced by Hagan et al. (2002), is a stochastic volatility model that assumes that volatility is a random process, like Heston. But, unlike Heston, the SABR model is particularly used to model forward prices, and is more particularly used in interest rates, FX, and commodities markets.

Remark: While the SABR model is mainly designed for forward price derivatives, in this thesis, we will use it for comparison of stochastic volatility modeling approaches. For simplicity in comparison, we will be using an option portfolio optimization, where we treat the spot price as the forward under risk-free rate (with zero dividend yield).

2.5.1 Model Assumptions

The SABR model assumes both the forward price F_t of the underlying asset and its volatility σ_t follow stochastic processes:

$$\begin{cases} dF_t = \sigma_t F_t^\beta dW_t^1 \\ d\sigma_t = \nu \sigma_t dW_t^2 \end{cases}$$

where:

- ν is the volatility of volatility ("vol-of-vol"), analogous to σ_v in the Heston model
- W_t^1 and W_t^2 are correlated Brownian motions with $\rho = \text{Corr}(dW_t^1, dW_t^2)$
- $\beta \in [0, 1]$ is the elasticity parameter, $\beta = 0$ gives the normal model, and $\beta = 1$ gives the lognormal model.
- $\beta \in [0, 1]$ is the elasticity parameter, which determines how proportional the volatility is to the forward price:
 - $\beta = 1$: volatility is proportional to the price (lognormal model)
 - $\beta = 0$: volatility is independent of the price (normal model)

– $0 < \beta < 1$: volatility grows with the price but less than proportionally

In commodity markets, the SABR model is typically implemented with $\beta = 1$, reflecting the lognormal nature of stock prices, as in Black-Scholes.

Unlike the Heston model, where variance follows a mean-reverting CIR process, the SABR model assumes that volatility follows a lognormal process *without mean reversion*, meaning that volatility shocks are more persistent than in Heston.

2.5.2 Approximate Pricing Formula

As with the Heston model, we work under the risk-neutral measure $\mathbb{Q}$, where the drift of the forward price is the risk-free rate:

$$\begin{cases} dF_t = \sigma_t F_t^\beta d\widetilde{W}_t^{(1)} \\ d\sigma_t = \nu \sigma_t d\widetilde{W}_t^{(2)} \end{cases}$$

The SABR model does not admit a closed-form solution for option prices. However, Hagan et al. (2002) derived a formula for implied volatility, that can be used in the Black-Scholes formula for pricing. For the implied volatility $\sigma_{BS}(K)$ of a European option with strike K , forward F , and maturity T , we have:

$$\sigma_{BS}(K) \approx \frac{\alpha(FK)^{(1-\beta)/2}}{1 + \frac{(1-\beta)^2}{24} [\log(F/K)]^2 + \frac{(1-\beta)^4}{1920} [\log(F/K)]^4} \frac{z}{x(z)}$$

where

$$z = \frac{\nu}{\alpha} (FK)^{(1-\beta)/2} \log(F/K), \quad x(z) = \log \left(\frac{\sqrt{1 - 2\rho z + z^2} + z - \rho}{1 - \rho} \right)$$

and α is the initial volatility (σ_0) and other parameters are as previously defined.

This formula is valid for a small time to maturity T and under the assumption that $F \neq K$. When $F = K$ (at-the-money), we have the following simplification:

$$\sigma_{BS}(F) \approx \frac{\alpha}{F^{1-\beta}} \left[1 + \left(\frac{(1-\beta)^2}{24} \frac{\alpha^2}{F^{2(1-\beta)}} + \frac{\rho\beta\nu\alpha}{4F^{1-\beta}} + \frac{2-3\rho^2}{24} \nu^2 \right) T \right]$$

2.5.3 Option Greeks under the SABR Model

The SABR model does not have closed-form formulas for option prices or their sensitivities. However, using Hagan's approximation of the implied volatility $\sigma_{BS}(K)$ within the Black-Scholes framework to compute option price, we can then compute the Greeks using

numerical differentiation.

Like in the Heston model, we apply **finite difference methods** to approximate the partial derivatives of the option price with respect to the inputs with a price evaluated at slightly perturbed values. Let $C(F, \alpha, T)$ be the European call price under the SABR model, computed using the SABR-implied volatility in the Black-Scholes formula:

$$C(F, \alpha, T) = BS(F, K, T, r, \sigma_{BS}(K)),$$

where $\sigma_{BS}(K)$ is the SABR-implied volatility. We approximate the Greeks as follows:

$$\begin{aligned}\text{Delta } \Delta &\approx \frac{C(F + \epsilon, \alpha, T) - C(F - \epsilon, \alpha, T)}{2\epsilon} \\ \text{Gamma } \Gamma &\approx \frac{C(F + \epsilon, \alpha, T) - 2C(F, \alpha, T) + C(F - \epsilon, \alpha, T)}{\epsilon^2} \\ \text{Vega } \nu &\approx \frac{C(F, \alpha + \epsilon, T) - C(F, \alpha - \epsilon, T)}{2\epsilon}\end{aligned}$$

for a small perturbation $\epsilon > 0$ in S (small enough to ensure an accurate approximation but not so small that numerical instability arises).

3 Methodology and Implementation

3.1 Selection of the Underlying Asset and the Study Period

As our options' underlying asset, we have chosen to work with WTI (West Texas Intermediate) Crude Oil for the following reasons:

- High historical volatility: Crude oil markets are characterized by sharp, sudden and frequent price changes, which is well-suited to test models like Heston and SABR
- Macroeconomic significance: WTI crude oil is a benchmark for global oil pricing, and is more generally linked to many macroeconomic factors (geopolitical tensions, monetary and fiscal policy, energy demand)

WTI crude oil is therefore particularly relevant in testing stochastic volatility models as its price undergoes both relatively calm periods and highly volatile ones. The time frame that we have selected runs from January 2015 to May 2025. This large study period captures several significant market environments (see Figure 2) :

- The 2014–2016 oil price collapse, driven by oversupply and slowing global growth
- The 2020 COVID-19 crisis, which led to a significant price decline as part of the unprecedented collapse of global demand
- The post-pandemic recovery



Figure 2: WTI Crude Oil Futures Prices (2015-2025)

Figure 2 displays the WTI Crude Oil Futures Prices. This is due to the lack of direct data on the WTI spot prices. So, instead of spot prices, we use front-month WTI futures (CL=F) as a proxy for spot prices. This approximation is acceptable since short-term futures closely reflect current market prices.

As such, we collected daily WTI crude oil data for the selected period from the Yahoo Finance financial provider. Figure 3 below shows a sample of the data we retrieved: in our study, we used the daily closing price as the price S_t .

Price Ticker Date	Close CL=F
2015-01-02	52.689999
2015-01-05	50.040001
2015-01-06	47.930000
2015-01-07	48.650002
2015-01-08	48.790001

Figure 3: Sample of WTI Crude Oil Prices in our dataset

3.2 Option Data Construction

Accurate historical data on crude oil options is particularly difficult to obtain. Due to this limited availability, we manually simulate option prices using the Black-Scholes model, generating a data set of option prices based on the underlying WTI crude oil prices collected earlier.

For each observation date in our selected sample period, we built a panel of 18 European options with varying maturities and levels of moneyness:

- Maturities T : short-term maturities * of 30 (1m), 90 (3m) and 180 days (6m).
 - 9 Calls: 3 calls at 30 days, 3 calls at 90 days, and 3 calls at 180 days
 - 9 Puts: 3 puts at 30 days, 3 puts at 90 days, and 3 puts at 180 days
- Strikes K : $\pm 10\%$ around the spot price S_t
 - 6 options Out of The Money (OTM): 3 Calls ($K > S_t$) and 3 Puts ($K < S_t$)
 - 6 options At The Money (ATM): 3 Calls ($K \approx S_t$) and 3 Puts ($K \approx S_t$)
 - 6 options In The Money (ITM): 3 Calls ($K < S_t$) and 3 Puts ($K > S_t$)

* Short maturities are assumed as it is needed for the SABR approximate pricing formula.

As previously mentioned, we used the Black-Scholes closed-form solutions for European option pricing to overcome the limitations of option price data availability. We recall that the formulas 8 are as follows:

$$C = S_0 N(d_1) - K e^{-rT} N(d_2), \quad P = K e^{-rT} N(-d_2) - S_0 N(-d_1)$$

where the terms d_1 and d_2 are given by:

$$d_1 = \frac{\ln\left(\frac{S_0}{K}\right) + \left(r + \frac{\sigma^2}{2}\right) T}{\sigma \sqrt{T}}, \quad d_2 = d_1 - \sigma \sqrt{T},$$

Having obtained the spot price S_t of Brent crude oil for our study period, and defined the maturity T and the strike K , we now need to make the following assumptions concerning the remaining model parameters to properly implement the Black-Scholes formulas:

- a constant volatility σ of 30% (historically ranging between 20-50% for crude oil)
- a risk-free rate r fixed at 2% (conventionally used in financial modelling)

In the following Figures 4, 5 and 6, we've computed the 18 European options prices on March 2nd 2020, January 4th 2021, and July 1st 2021 with the Black-Scholes formula. Our reasons for choosing these three dates as starting points will become more apparent in later analyses, specifically during the backtests.

	Date	Type	K (strike)	T (days)	S (spot)	Price (BSM)
0	2020-03-02	call	42.075	30	46.75	4.939165
1	2020-03-02	put	42.075	30	46.75	0.195058
2	2020-03-02	call	46.750	30	46.75	1.640958
3	2020-03-02	put	46.750	30	46.75	1.564172
4	2020-03-02	call	51.425	30	46.75	0.295273
5	2020-03-02	put	51.425	30	46.75	4.885809
6	2020-03-02	call	42.075	90	46.75	5.759534
7	2020-03-02	put	42.075	90	46.75	0.877551
8	2020-03-02	call	46.750	90	46.75	2.885465
9	2020-03-02	put	46.750	90	46.75	2.655485
10	2020-03-02	call	51.425	90	46.75	1.213846
11	2020-03-02	put	51.425	90	46.75	5.635868
12	2020-03-02	call	42.075	180	46.75	6.790988
13	2020-03-02	put	42.075	180	46.75	1.703041
14	2020-03-02	call	46.750	180	46.75	4.136362
15	2020-03-02	put	46.750	180	46.75	3.677533
16	2020-03-02	call	51.425	180	46.75	2.342664
17	2020-03-02	put	51.425	180	46.75	6.512952

Figure 4: Crude Oil Options generated using Black-Scholes (2020-03-02)

	Date	Type	K (strike)	T (days)	S (spot)	Price (BSM)
0	2021-01-04	call	42.857999	30	47.619999	5.031081
1	2021-01-04	put	42.857999	30	47.619999	0.198688
2	2021-01-04	call	47.619999	30	47.619999	1.671496
3	2021-01-04	put	47.619999	30	47.619999	1.593281
4	2021-01-04	call	52.381999	30	47.619999	0.300768
5	2021-01-04	put	52.381999	30	47.619999	4.976731
6	2021-01-04	call	42.857999	90	47.619999	5.866716
7	2021-01-04	put	42.857999	90	47.619999	0.893882
8	2021-01-04	call	47.619999	90	47.619999	2.939163
9	2021-01-04	put	47.619999	90	47.619999	2.704902
10	2021-01-04	call	52.381999	90	47.619999	1.236435
11	2021-01-04	put	52.381999	90	47.619999	5.740749
12	2021-01-04	call	42.857999	180	47.619999	6.917365
13	2021-01-04	put	42.857999	180	47.619999	1.734734
14	2021-01-04	call	47.619999	180	47.619999	4.213338
15	2021-01-04	put	47.619999	180	47.619999	3.745970
16	2021-01-04	call	52.381999	180	47.619999	2.386260
17	2021-01-04	put	52.381999	180	47.619999	6.634155

Figure 5: Crude Oil Options generated using Black-Scholes (2021-01-04)

	Date	Type	K (strike)	T (days)	S (spot)	Price (BSM)
0	2021-07-01	Call	67.707003	30	75.230003	7.948095
1	2021-07-01	Put	67.707003	30	75.230003	0.313887
2	2021-07-01	Call	75.230003	30	75.230003	2.640627
3	2021-07-01	Put	75.230003	30	75.230003	2.517063
4	2021-07-01	Call	82.753004	30	75.230003	0.475153
5	2021-07-01	Put	82.753004	30	75.230003	7.862233
6	2021-07-01	Call	67.707003	90	75.230003	9.268230
7	2021-07-01	Put	67.707003	90	75.230003	1.412154
8	2021-07-01	Call	75.230003	90	75.230003	4.643285
9	2021-07-01	Put	75.230003	90	75.230003	4.273201
10	2021-07-01	Call	82.753004	90	75.230003	1.953318
11	2021-07-01	Put	82.753004	90	75.230003	9.069226
12	2021-07-01	Call	67.707003	180	75.230003	10.928044
13	2021-07-01	Put	67.707003	180	75.230003	2.740531
14	2021-07-01	Call	75.230003	180	75.230003	6.656226
15	2021-07-01	Put	75.230003	180	75.230003	5.917878
16	2021-07-01	Call	82.753004	180	75.230003	3.769811
17	2021-07-01	Put	82.753004	180	75.230003	10.480629

Figure 6: Crude Oil Options generated using Black-Scholes (2021-07-01)

3.3 Portfolio Objectives and Optimization Constraints

Our central objective is to build a portfolio of options that are insensitive to the underlying crude oil price movements, while being highly sensitive to changes in volatility. This allows us to isolate and compare the Heston and SABR models purely on their capability in predicting and exploiting volatility, without being influenced by directional market movements.

To do so, the optimization constraints we've set are Delta and Gamma neutrality. Delta neutrality eliminates exposure to linear movements in the underlying asset, while Gamma neutrality eliminates sensitivity to its convexity. This isolates the effect of volatility on the portfolio, captured by Vega.

The optimization process then consists in maximizing the total Vega of the portfolio, meaning increasing its overall exposure to changes in volatility. Although we don't explicitly constrain Vega, we will later use PnL per unit of Vega to assess risk-adjusted returns, since Vega varies significantly across portfolios.

Our portfolio optimization problem can be seen as follows:

$$\begin{aligned}
&\text{Maximize: } \sum_{i=1}^n w_i \cdot \nu_i \quad (\text{Total Vega}) \\
&\text{Subject to:} \\
&\quad \sum_{i=1}^n w_i = 1 \quad (\text{Budget constraint}) \\
&\quad -1 \leq w_i \leq 1 \quad (\text{Position limits}) \\
&\quad |\Gamma^P| \leq 0.30 \quad (\text{Gamma neutrality}) \\
&\quad \Delta^P = 0.00 \quad (\text{Delta neutrality})
\end{aligned}$$

where w_i is the weight of option i and ν_i its Vega. We chose a threshold of 0.30 for Gamma because it is close enough to zero to approximate Gamma neutrality, yet it allows us to avoid overconstraining the optimization.

We initially considered having a Gamma constraint on each option too ($|w_i \cdot \Gamma_i| \leq 0.3$). However, we later observed that this severely restricted our optimization, which led to less relevant results in comparing portfolio performance. Moreover, from a trading perspective, in practice, constraints on Gamma are typically put at the portfolio level.

3.4 Implementation and Results of the Optimization

Now that the optimization objectives have been set, we can proceed with the implementation of the predefined optimization procedure for each model. We currently have three portfolios, each composed of 18 European options with the same features, except for the prices, which were previously calculated with the spot and strike prices corresponding to three reference dates: March 2nd 2020, January 4th 2021 and July 1st 2021.

3.4.1 Optimization under the SABR Model

To price options under the SABR model and compute their Greeks, we used the Hagan approximation (section 2.5.2) to compute the implied volatility. And to prevent too divergent volatilities, we applied a lower bound.

Second, we calibrated the SABR parameters (α, ρ, ν) to implied volatilities derived from Black-Scholes prices of call options, minimizing the squared error between market-implied (by Black-Scholes in our case) and SABR model-implied volatilities. Constraints on the bounds of the parameters are imposed, with $-1 \leq \rho \leq 1$, and $0.001 \leq \alpha, \nu \leq 2$ as reasonable bounds that prevent division by zero. The β parameter is fixed to 0.5.

Then, once calibrated, the SABR-implied volatility is used as input into the Black-Scholes

formula to derive the SABR price of each option and to compute Delta, Gamma and Vega using the finite differences methods (section 2.5.3) with $\varepsilon = 10^{-4}$).

Finally, applying the optimization objectives, we get the weights and Greeks for each option in the three portfolios. Figure 7, 8 and 9 show that the three optimized portfolios correctly respect the optimization constraints we had set, with $\Delta = 0$, $|\Gamma| \leq 0.3$, and $\sum w_i = 1$ with $-1 \leq w_i \leq 1$.

	Date	Type	S (spot)	K (strike)	T (days)	Delta (SABR)	Gamma (SABR)	Vega (SABR)	Weight (SABR)
0	2020-03-02	Call	46.75	42.075	30	0.918053	0.035107	2.335367	-1.000000
1	2020-03-02	Put	46.75	42.075	30	-0.081947	0.035107	2.335367	-1.000000
2	2020-03-02	Call	46.75	46.750	30	0.541443	1.000000	3.819706	1.000000
3	2020-03-02	Put	46.75	46.750	30	-0.458557	1.000000	3.819706	1.000000
4	2020-03-02	Call	46.75	51.425	30	0.142634	0.057302	3.093856	-1.000000
5	2020-03-02	Put	46.75	51.425	30	-0.857366	0.057302	3.093856	-1.000000
6	2020-03-02	Call	46.75	42.075	90	0.841011	0.036242	6.644831	-0.032931
7	2020-03-02	Put	46.75	42.075	90	-0.158989	0.036241	6.644831	-0.606296
8	2020-03-02	Call	46.75	46.750	90	0.571618	1.000000	3.376593	0.086202
9	2020-03-02	Put	46.75	46.750	90	-0.428382	1.000000	3.376593	-0.446976
10	2020-03-02	Call	46.75	51.425	90	0.283946	0.054283	8.038225	1.000000
11	2020-03-02	Put	46.75	51.425	90	-0.716054	0.054283	8.038225	1.000000
12	2020-03-02	Call	46.75	42.075	180	0.820309	0.032382	10.587572	1.000000
13	2020-03-02	Put	46.75	42.075	180	-0.179691	0.032382	10.587572	1.000000
14	2020-03-02	Call	46.75	46.750	180	0.600939	1.000000	1.741508	-1.000000
15	2020-03-02	Put	46.75	46.750	180	-0.399061	1.000000	1.741508	-1.000000
16	2020-03-02	Call	46.75	51.425	180	0.361203	0.047715	12.520030	1.000000
17	2020-03-02	Put	46.75	51.425	180	-0.638797	0.047714	12.520030	1.000000
Total Delta : 0.0000									
Total Gamma : -0.3000									
Total Vega : 50.1239									
Sum of weights: 1.0000									

Figure 7: Optimized portfolio using the SABR model (2020-03-02)

	Date	Type	S (spot)	K (strike)	T (days)	Delta (SABR)	Gamma (SABR)	Vega (SABR)	Weight (SABR)
0	2021-01-04	Call	47.619999	42.857999	30	0.917435	0.034718	2.378835	-1.000000
1	2021-01-04	Put	47.619999	42.857999	30	-0.082565	0.034719	2.378835	-1.000000
2	2021-01-04	Call	47.619999	47.619999	30	0.541477	1.000000	3.890789	1.000000
3	2021-01-04	Put	47.619999	47.619999	30	-0.458523	1.000000	3.890789	1.000000
4	2021-01-04	Call	47.619999	52.381999	30	0.143599	0.056648	3.151423	-1.000000
5	2021-01-04	Put	47.619999	52.381999	30	-0.856401	0.056648	3.151423	-1.000000
6	2021-01-04	Call	47.619999	42.857999	90	0.839259	0.035681	6.768493	-0.036728
7	2021-01-04	Put	47.619999	42.857999	90	-0.160741	0.035682	6.768493	-0.605227
8	2021-01-04	Call	47.619999	47.619999	90	0.571675	1.000000	3.439430	0.083217
9	2021-01-04	Put	47.619999	47.619999	90	-0.428325	1.000000	3.439430	-0.441262
10	2021-01-04	Call	47.619999	52.381999	90	0.286447	0.053425	8.187808	1.000000
11	2021-01-04	Put	47.619999	52.381999	90	-0.713553	0.053426	8.187808	1.000000
12	2021-01-04	Call	47.619999	42.857999	180	0.817529	0.031708	10.784604	1.000000
13	2021-01-04	Put	47.619999	42.857999	180	-0.182471	0.031709	10.784604	1.000000
14	2021-01-04	Call	47.619999	47.619999	180	0.601017	1.000000	1.773917	-1.000000
15	2021-01-04	Put	47.619999	47.619999	180	-0.398983	1.000000	1.773917	-1.000000
16	2021-01-04	Call	47.619999	52.381999	180	0.365080	0.046708	12.753021	1.000000
17	2021-01-04	Put	47.619999	52.381999	180	-0.634920	0.046708	12.753021	1.000000
Total Delta : -0.0000									
Total Gamma : -0.3000									
Total Vega : 51.0476									
Sum of weights: 1.0000									

Figure 8: Optimized portfolio using the SABR model (2021-01-04)

	Date	Type	S (spot)	K (strike)	T (days)	Delta (SABR)	Gamma (SABR)	Vega (SABR)	Weight (SABR)
0	2021-07-01	Call	75.230003	67.707003	30	0.910285	0.023768	3.758059	-1.000000
1	2021-07-01	Put	75.230003	67.707003	30	-0.089715	0.023765	3.758059	-1.000000
2	2021-07-01	Call	75.230003	75.230003	30	0.541846	1.000000	6.146663	1.000000
3	2021-07-01	Put	75.230003	75.230003	30	-0.458154	1.000000	6.146663	1.000000
4	2021-07-01	Call	75.230003	82.753004	30	0.154818	0.038564	4.978629	-1.000000
5	2021-07-01	Put	75.230003	82.753004	30	-0.845182	0.038563	4.978629	-1.000000
6	2021-07-01	Call	75.230003	67.707003	90	0.818909	0.023000	10.692840	-0.069107
7	2021-07-01	Put	75.230003	67.707003	90	-0.181091	0.023001	10.692840	-0.610190
8	2021-07-01	Call	75.230003	75.230003	90	0.572313	1.000000	5.433606	0.036853
9	2021-07-01	Put	75.230003	75.230003	90	-0.427687	1.000000	5.433606	-0.357556
10	2021-07-01	Call	75.230003	82.753004	90	0.315604	0.034148	12.935096	1.000000
11	2021-07-01	Put	75.230003	82.753004	90	-0.684396	0.034148	12.935096	1.000000
12	2021-07-01	Call	75.230003	67.707003	180	0.785089	0.018876	17.037491	1.000000
13	2021-07-01	Put	75.230003	67.707003	180	-0.214911	0.018877	17.037491	1.000000
14	2021-07-01	Call	75.230003	75.230003	180	0.601911	1.000000	2.802431	-1.000000
15	2021-07-01	Put	75.230003	75.230003	180	-0.398089	1.000000	2.802431	-1.000000
16	2021-07-01	Call	75.230003	82.753004	180	0.410516	0.027469	20.000000	1.000000
17	2021-07-01	Put	75.230003	82.753004	180	-0.589484	0.027469	20.000000	1.000000
Total Delta : 0.0000									
Total Gamma : -0.3000									
Total Vega : 80.1541									
Sum of weights: 1.0000									

Figure 9: Optimized portfolio using the SABR model (2021-07-01)

3.4.2 Optimization under the Heston Model

To compute option prices under the Heston Model, we put in place a numerical integration of the semi-closed pricing formula using the characteristic function approach. We chose the following parameters: initial variance $v_0 = 0.04$, long-term variance $\theta = 0.04$, speed of mean reversion $\kappa = 1.5$, volatility of volatility $\sigma = 0.3$, and correlation $\rho = -0.7$. Then, the Greeks are computed with the finite differences method (section 2.4.5) with $\varepsilon = 10^{-4}$.

Finally, applying the optimization objectives, we get the weights and Greeks for each option in the three portfolios. Figure 10, 11 and 12 show that the three optimized portfolios correctly respect the optimization constraints we had set, with $\Delta = 0$, $|\Gamma| \leq 0.3$, and $\sum w_i = 1$ with $-1 \leq w_i \leq 1$.

	Date	Type	S (spot)	K (strike)	T (days)	Delta (Heston)	Gamma (Heston)	Vega (Heston)	Weight (Heston)
0	2020-03-02	Call	46.75	42.075	30	-0.547860	0.613325	20.000000	-1.000000
1	2020-03-02	Put	46.75	42.075	30	-1.547860	0.613325	20.000000	-1.000000
2	2020-03-02	Call	46.75	46.750	30	-6.448206	0.879835	15.792196	-1.000000
3	2020-03-02	Put	46.75	46.750	30	-7.448206	0.879835	15.792196	0.445097
4	2020-03-02	Call	46.75	51.425	30	-1.546119	-1.631150	-20.000000	0.143650
5	2020-03-02	Put	46.75	51.425	30	-2.546119	-1.631150	-20.000000	1.000000
6	2020-03-02	Call	46.75	42.075	90	-0.970781	0.389084	20.000000	-0.588748
7	2020-03-02	Put	46.75	42.075	90	-1.970781	0.389084	20.000000	1.000000
8	2020-03-02	Call	46.75	46.750	90	-3.500392	0.497742	20.000000	1.000000
9	2020-03-02	Put	46.75	46.750	90	-4.500392	0.497742	20.000000	1.000000
10	2020-03-02	Call	46.75	51.425	90	-3.441287	-0.700345	-20.000000	-1.000000
11	2020-03-02	Put	46.75	51.425	90	-4.441287	-0.700345	-20.000000	-1.000000
12	2020-03-02	Call	46.75	42.075	180	-0.819383	0.240740	20.000000	1.000000
13	2020-03-02	Put	46.75	42.075	180	-1.819383	0.240740	20.000000	1.000000
14	2020-03-02	Call	46.75	46.750	180	-2.326348	0.335480	19.731042	1.000000
15	2020-03-02	Put	46.75	46.750	180	-3.326348	0.335480	19.731042	1.000000
16	2020-03-02	Call	46.75	51.425	180	-3.174189	-0.085856	-20.000000	-1.000000
17	2020-03-02	Put	46.75	51.425	180	-4.174189	-0.085856	-20.000000	-1.000000
Total Delta : 0.0000									
Total Gamma : 0.3000									
Total Vega : 136.0510									
Sum of weights: 1.0000									

Figure 10: Optimized portfolio using the Heston model (2020-03-02)

	Date	Type	S (spot)	K (strike)	T (days)	Delta (Heston)	Gamma (Heston)	Vega (Heston)	Weight (Heston)
0	2021-01-04	Call	47.619999	42.857999	30	-0.547860	0.602127	20.000000	-1.000000
1	2021-01-04	Put	47.619999	42.857999	30	-1.547860	0.602127	20.000000	-1.000000
2	2021-01-04	Call	47.619999	47.619999	30	-6.448206	0.863767	16.086083	-1.000000
3	2021-01-04	Put	47.619999	47.619999	30	-7.448206	0.863767	16.086083	0.445338
4	2021-01-04	Call	47.619999	52.381999	30	-1.546119	-1.601346	-20.000000	0.140941
5	2021-01-04	Put	47.619999	52.381999	30	-2.546119	-1.601346	-20.000000	1.000000
6	2021-01-04	Call	47.619999	42.857999	90	-0.970781	0.381981	20.000000	-0.586279
7	2021-01-04	Put	47.619999	42.857999	90	-1.970781	0.381981	20.000000	1.000000
8	2021-01-04	Call	47.619999	47.619999	90	-3.500392	0.488650	20.000000	1.000000
9	2021-01-04	Put	47.619999	47.619999	90	-4.500392	0.488650	20.000000	1.000000
10	2021-01-04	Call	47.619999	52.381999	90	-3.441287	-0.687547	-20.000000	-1.000000
11	2021-01-04	Put	47.619999	52.381999	90	-4.441287	-0.687547	-20.000000	-1.000000
12	2021-01-04	Call	47.619999	42.857999	180	-0.819383	0.236343	20.000000	1.000000
13	2021-01-04	Put	47.619999	42.857999	180	-1.819383	0.236343	20.000000	1.000000
14	2021-01-04	Call	47.619999	47.619999	180	-2.326348	0.329354	20.000000	1.000000
15	2021-01-04	Put	47.619999	47.619999	180	-3.326348	0.329354	20.000000	1.000000
16	2021-01-04	Call	47.619999	52.381999	180	-3.174189	-0.084287	-20.000000	-1.000000
17	2021-01-04	Put	47.619999	52.381999	180	-4.174189	-0.084287	-20.000000	-1.000000
Total Delta : -0.0000									
Total Gamma : 0.3000									
Total Vega : 136.5333									
Sum of weights: 1.0000									

Figure 11: Optimized portfolio using the Heston model (2021-01-04)

	Date	Type	S (spot)	K (strike)	T (days)	Delta (Heston)	Gamma (Heston)	Vega (Heston)	Weight (Heston)
0	2021-07-01	Call	75.230003	67.707003	30	-0.547860	0.381128	20.0	-1.000000
1	2021-07-01	Put	75.230003	67.707003	30	-1.547860	0.381128	20.0	-1.000000
2	2021-07-01	Call	75.230003	75.230003	30	-6.448206	0.546723	20.0	-1.000000
3	2021-07-01	Put	75.230003	75.230003	30	-7.448206	0.546723	20.0	0.452964
4	2021-07-01	Call	75.230003	82.753004	30	-1.546119	-1.013657	-20.0	0.055086
5	2021-07-01	Put	75.230003	82.753004	30	-2.546119	-1.013657	-20.0	1.000000
6	2021-07-01	Call	75.230003	67.707003	90	-0.970781	0.241781	20.0	-0.508049
7	2021-07-01	Put	75.230003	67.707003	90	-1.970781	0.241781	20.0	1.000000
8	2021-07-01	Call	75.230003	75.230003	90	-3.500392	0.309293	20.0	1.000000
9	2021-07-01	Put	75.230003	75.230003	90	-4.500392	0.309293	20.0	1.000000
10	2021-07-01	Call	75.230003	82.753004	90	-3.441287	-0.435231	-20.0	-1.000000
11	2021-07-01	Put	75.230003	82.753004	90	-4.441287	-0.435230	-20.0	-1.000000
12	2021-07-01	Call	75.230003	67.707003	180	-0.819383	0.149589	20.0	1.000000
13	2021-07-01	Put	75.230003	67.707003	180	-1.819383	0.149589	20.0	1.000000
14	2021-07-01	Call	75.230003	75.230003	180	-2.326348	0.208458	20.0	1.000000
15	2021-07-01	Put	75.230003	75.230003	180	-3.326348	0.208458	20.0	1.000000
16	2021-07-01	Call	75.230003	82.753004	180	-3.174189	-0.053372	-20.0	-1.000000
17	2021-07-01	Put	75.230003	82.753004	180	-4.174189	-0.053372	-20.0	-1.000000
Total Delta : -0.0000									
Total Gamma : 0.3000									
Total Vega : 137.7966									
Sum of weights: 1.0000									

Figure 12: Optimized portfolio using the Heston model (2021-07-01)

3.5 Backtesting and Performance evaluation

3.5.1 Backtesting Periods

Up until now, we had constructed three portfolios with three different starting dates. Now, each portfolio corresponds to a specific backtesting period. The three backtesting periods we have selected each represent a unique market regime. This allows us to properly evaluate the performance of each stochastic volatility model. The selected periods are as follows:

- **March 2020 – September 2020:** (*March 2nd start date*) A period with extreme volatility and volatility clustering in the context of the COVID-19 crisis. This period will assess how well the models can manage high volatility regimes.
 - Volatility clustering: empirical observation that high (resp. low) periods of volatility tend to be followed by more periods of high (resp. low) volatility
- **January 2021 – July 2021:** (*January 4th start date*) A period in which volatility is moderate compared to the first period, but skew and smiles remained pronounced. This post-crisis period will assess how well the models manage volatility asymmetries.
 - Volatility skew: phenomenon in which implied volatility varies with strike prices, which creates asymmetries in the volatility curve
- **July 2021 – January 2022:** (*July 1st start date*) A relatively calm period with significantly lower volatility. This period will assess how well the models function under quasi-normal conditions.

These three periods were selected intentionally to have diverse market dynamics and assess whether the Greeks-based portfolio optimization strategies perform consistently across regimes. In each period, portfolios were optimized on the starting date, and their performance was compared 6 months after the starting date, to align with the options with the longest maturities (180 days). The starting dates were chosen as the first date of the month with available data on the underlying asset price.

3.5.2 Performance Evaluation Metrics

To compare the performance of the option portfolios optimized under Heston and SABR, we have selected two key indicators:

- Profit and Loss * :

$$\text{PnL} = \sum_{i=1}^n w_i (\text{Payoff}_i - \text{Price}_i^{\text{initial}})$$

- PnL per unit of Vega:

$$\text{PnL/Vega} = \frac{\text{PnL}}{\sum_{i=1}^n |w_i \cdot \text{Vega}_i|}$$

* We consider a 1 euro portfolio.

PnL measures the raw performance of the portfolio for each period, while the PnL per unit of Vega gives us a risk-adjusted performance, by taking into account the portfolio's exposure to volatility. The best-performing model in a given context will be the one that achieves the highest PnL and PnL per unit of Vega.

4 Results and Discussion

4.1 Empirical Findings

Computing our two performance metrics under the Heston and SABR models across our three backtesting periods, we get the following results. Figure 13 compares the total Profit and Loss, while Figure 14 normalizes the performance by Vega exposure.

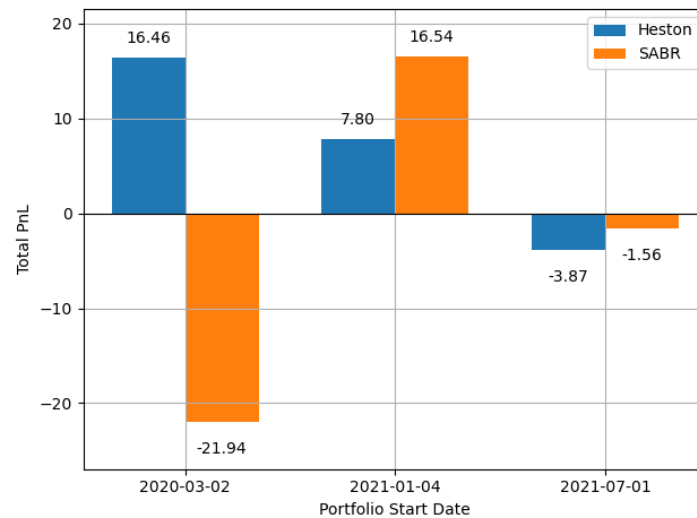


Figure 13: Profit and Loss: Heston and SABR

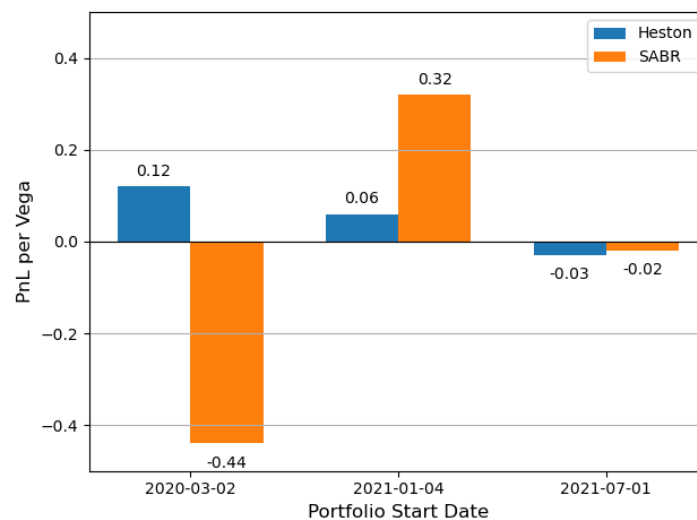


Figure 14: Profit and Loss per unit of Vega: Heston and SABR

Summarizing the results by market conditions, we can determine under which conditions each greek-based model optimization performs the best.

Period	Market Conditions	Better Model
2020-03 to 2020-09	Extreme volatility: volatility clustering	Heston
2021-01 to 2021-07	Decreasing volatility in a recovering market: volatility skews	SABR
2021-07 to 2022-01	Low and stable volatility	Heston/SABR

Table 3: Model Comparison Across Market Conditions

2020-03 to 2020-09 Under high volatility and volatility clustering, the Heston model, with its mean-reverting variance, performed better than the SABR model, providing both a higher PnL (16.46 EUR > -21.94 EUR) and a higher PnL per unit of Vega (0.12 EUR/Vega > -0.44 EUR/Vega)

2021-01 to 2021-07 The SABR model performed better during this recovery phase. This phase was marked by decreasing volatility and volatility skews. The SABR model indeed captured the skews effects it was designed to handle quite well. While Heston still performed well in terms of PnL (7.8 EUR), SABR provided better Vega efficiency (0.32 EUR/Vega < 0.06 EUR/Vega).

2021-07 to 2022-01 Under almost normal conditions, the SABR model slightly outperformed Heston, but neither model delivered particularly high returns, which suggests lower opportunity for profits in this kind of environment with stochastic volatility models.

4.2 Critical Reflections

Our choice to use Black-Scholes as a pricing tool, despite its limits in reflecting real-world markets due to its idealized assumptions was not a major issue here. Indeed, our objective was to compare the quality of the Greeks the SABR and Heston model generate. The use of Black-Scholes to price our initial options simply serves as a common and coherent base on which we can apply our two relevant stochastic volatility models.

However, we did observe that, under certain parameter choices, the option prices generated by the Heston model were significantly higher than the Black-Scholes ones. And, choosing the parameters as to best be in sync with the Black-Scholes prices, we still encountered a higher gap than what normally should have been.

Finally, though our optimization under both models respected the given constraints, the results were not smooth, with many weights pushed at the bounds $([-1,1])$. This likely signaled that our optimization was too constrained. Moreover, our optimization neglected transaction costs and liquidity constraints, both of which are crucial in real-world applications.

5 Conclusion

This thesis set out to evaluate the performance of the Heston and SABR models, two widely used stochastic volatility models, in the context of option portfolio construction and optimization under Greek-based constraints.

Using Black-Scholes to simulate the market prices of European options, we overcame the issue of data availability. Then, we built a panel of three portfolios with diverse options, all of which were optimized according to our predefined objectives based on the Greeks we computed both under the Heston and SABR models following the finite difference method. Our portfolios were then tested across three backtesting periods, each characterized by specific market volatility conditions.

Our results revealed that no single model outperformed in all market conditions, both in terms of profit and loss (PnL) and risk-adjusted returns (PnL per unit of Vega). The Heston model achieved the highest risk-adjusted returns under extreme volatility regimes. In assuming mean-reverting variance, it was the most suitable model in handling volatility clustering. On the other hand, the SABR best captured the skew effects during the recovery phase. And, under calmer market conditions, no model displayed a significantly better performance.

However, we acknowledged that our experimental framework did not fully reflect real-world market conditions, with several limitations that need to be taken into account.

Overall, this study illustrates the practical implications of choosing a stochastic volatility model in optimizing a given portfolio. It corroborates several well-established characteristics of the Heston and SABR models, and shows the importance of understanding the current market conditions in order to achieve the best option portfolio performance.

References

- [1] John C. Hull, *Options, Futures, and Other Derivatives*, 11th Edition, Pearson, 2022.
- [2] Fischer Black and Myron Scholes, *The Pricing of Options and Corporate Liabilities*, Journal of Political Economy, Vol. 81, No. 3, pp. 637–654, 1973.
- [3] Robert C. Merton, *Theory of Rational Option Pricing*, Bell Journal of Economics and Management Science, Vol. 4, No. 1, pp. 141–183, 1973.
- [4] Steven E. Shreve, *Stochastic Calculus for Finance II: Continuous-Time Models*, Springer, 2004.
- [5] Steven L. Heston, *A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options*, The Review of Financial Studies, Vol. 6, No. 2, 1993, pp. 327–343.
- [6] Paul Wilmott, *Paul Wilmott on Quantitative Finance*, 2nd Edition, Wiley, 2006.
- [7] Natalia V. Batalova, Vassili Maroussov, and Frederi G. Viens, *Selection of an Optimal Portfolio with Stochastic Volatility and Discrete Observations*, 2006.
- [8] Patrick S. Hagan, Deep Kumar, Andrew S. Lesniewski, and Diana E. Woodward, *Managing Smile Risk*, Wilmott Magazine, September 2002, pp. 84–108.

Appendix

```
import yfinance as yf
import numpy as np
import pandas as pd
from scipy.stats import norm
from datetime import datetime
from scipy.optimize import brentq, minimize
from scipy.integrate import quad
import matplotlib.pyplot as plt

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 200)

#Plotting Payoffs

K = 100
S = np.linspace(50, 150, 200)
premium = 0
long_call = np.maximum(S - K, 0) - premium
short_call = -long_call
long_put = np.maximum(K - S, 0) - premium
short_put = -long_put
fig, axs = plt.subplots(2, 2, figsize=(10, 6))
fig.suptitle('Option Payoff Diagrams at Maturity', fontsize=14)
titles = ['Long Call', 'Short Call', 'Long Put', 'Short Put']
payoffs = [long_call, short_call, long_put, short_put]
colors = ['blue', 'red', 'green', 'orange']
for ax, title, payoff, color in zip(axs.flat, titles, payoffs, colors):
    ax.plot(S, payoff, label=title, color=color)
    ax.axhline(0, color='black', linewidth=0.8)
    ax.axvline(K, color='gray', linestyle='--', label='Strike Price K')
    ax.set_title(title)
    ax.set_xlabel('Underlying Price $S_T$')
    ax.set_ylabel('Payoff')
    ax.grid(True)
    ax.legend()

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.savefig("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/payoff.png")
plt.show()

# Retrieving data

wtiprices = yf.download("CL=F", start="2015-01-01", end="2025-05-31")[['Close']]
wtiprices = wtiprices[['Close']]
wtiprices.to_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/
                wti_prices.csv")
print(wtiprices.head())

# BSM Pricing

r = 0.02
sigma_const = 0.3
spot_dates = ["2020-03-02", "2021-01-04", "2021-07-01"]
maturities = [30, 90, 180]
strikes_per = [-0.1, 0, 0.1]
option_types = ['Call', 'Put']
```

```

beta = 0.5

wtiprices = pd.read_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/
    wti_prices.csv", skiprows=2)
wtiprices.columns = ['Date', 'Close']
wtiprices['Date'] = pd.to_datetime(wtiprices['Date'])
wtiprices.set_index('Date', inplace=True)

def bsm_price(S, K, T, r, sigma, option_type):
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2)*T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    if option_type == "Call":
        price = S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
    else:
        price = K * np.exp(-r * T) * norm.cdf(-d2) - S * norm.cdf(-d1)
    return price

def options(date):
    S = float(wtiprices.loc[date]['Close'])
    options = []
    for T in maturities:
        for strike_per in strikes_per:
            K = S * (1 + strike_per)
            for option_type in option_types:
                price = bsm_price(S, K, T/365, 0.02, 0.3, option_type)
                options.append({
                    'Date': date,
                    'Type': option_type,
                    'K (strike)': K,
                    'T (days)': T,
                    'S (spot)': S,
                    'Price (BSM)': price,
                })
    return pd.DataFrame(options)

options1 = options("2020-03-02")
options1.to_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/options1.
    csv")
options2 = options("2021-01-04")
options2.to_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/options2.
    csv")
options3 = options("2021-07-01")
options3.to_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/options3.
    csv")

#SABR Pricing and Greeks

def sabr_vol(S, K, T, alpha, beta, rho, nu):
    if S == K:
        vol = (alpha / S**(1 - beta))*(1 + (((1 - beta)**2)/24)*((alpha**2)/S**(2*(1-
            beta))))+(rho*beta*nu*alpha)/(4*(S**(1-beta)))+((2-3*(rho**2))/24)*(nu**2))*
            T)
    else:
        z = (nu/alpha) * (S*K)**((1 - beta)/2) * np.log(S/K)
        x_z = np.log((np.sqrt(1 - 2*rho*z + z**2) + z - rho) / (1 - rho))
        denom = ((1 + (((1 - beta)**2)/24)*(np.log(S/K)**2) + (((1 - beta)**4)/1920)*(
            np.log(S/K)**4)) * x_z)
        vol = (alpha * (S*K)**((1 - beta)/2) * z) / denom if denom != 0 else np.nan

```

```

    return max(vol, 0.007)

def calibrate_sabr(F, K_data, vol_data, T):
    def sabr_errors(params):
        alpha, rho, nu = params
        errors = []
        for K, market_vol in zip(K_data, vol_data):
            model_vol = sabr_vol(F, K, T, alpha, beta, rho, nu)
            errors.append((market_vol - model_vol)**2)
        return sum(errors)
    start = [0.2, 0.0, 0.5]
    bounds = [(0.001, 2), (-0.999, 0.999), (0.001, 2)]
    result = minimize(sabr_errors, x0=start, bounds=bounds)
    return result.x

def sabr_greeks(S, K, T, r, alpha, beta, rho, nu, option_type, eps=1e-4):
    sigma = sabr_vol(S, K, T, alpha, beta, rho, nu)
    price_base = bsm_price(S, K, T, r, sigma, option_type)
    price_plus = bsm_price(S + eps, K, T, r, sabr_vol(S + eps, K, T, alpha, beta, rho,
        nu), option_type)
    price_minus = bsm_price(S - eps, K, T, r, sabr_vol(S - eps, K, T, alpha, beta, rho
        , nu), option_type)
    delta = (price_plus - price_minus) / (2 * eps)
    gamma = (price_plus - 2 * price_base + price_minus) / (eps ** 2)
    vega = (bsm_price(S, K, T, r, sigma + eps, option_type) -
        bsm_price(S, K, T, r, max(sigma - eps, 1e-6), option_type)) / (2 * eps)
    return delta, gamma, vega

def options_with_sabr(date_str):
    S = float(wtiprices.loc[date_str]['Close'])
    results = []
    for T_days in maturities:
        T = T_days / 365
        strikes = [S * (1 + p) for p in strikes_per]
        bsm_calls = [bsm_price(S, K, T, r, sigma_const, "Call") for K in strikes]
        alpha, rho, nu = calibrate_sabr(S, strikes, [implied_volatility_call(p, S, K, T,
            r, "Call") for p, K in zip(bsm_calls, strikes)], T)
        for K in strikes:
            for opt_type in option_types:
                vol_sabr = sabr_vol(S, K, T, alpha, beta, rho, nu)
                price_bsm = bsm_price(S, K, T, r, sigma_const, opt_type)
                price_sabr = bsm_price(S, K, T, r, vol_sabr, opt_type)
                sabr_delta, sabr_gamma, sabr_vega = sabr_greeks(S, K, T, r, alpha, beta
                    , rho, nu, opt_type)
                results.append({
                    'Date': date_str,
                    'Type': opt_type,
                    'K (strike)': K,
                    'T (days)': T_days,
                    'S (spot)': S,
                    'Price (BSM)': price_bsm,
                    'Price (SABR)': price_sabr,
                    'Vol (SABR)': vol_sabr,
                    'Delta (SABR)': sabr_delta,
                    'Gamma (SABR)': sabr_gamma,
                    'Vega (SABR)': sabr_vega
                })
    return pd.DataFrame(results)

```



```

options1 = options_with_sabr("2020-03-02")
options1.to_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/options1.
    csv")
options2 = options_with_sabr("2021-01-04")
options2.to_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/options2.
    csv")
options3 = options_with_sabr("2021-07-01")
options3.to_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/options3.
    csv")

```

#Heston Pricing and Greeks

```

def heston_char(u, S, K, T, r, v0, theta, kappa, sigma, rho, option_type):
    a = kappa * theta
    b = kappa
    d = np.sqrt((rho * sigma * 1j * u - b)**2 + (sigma**2) * (1j * u + u**2))
    g = (b - rho * sigma * 1j * u + d) / (b - rho * sigma * 1j * u - d)
    C = (r * 1j * u * T +
        a / sigma**2 * ((b - rho * sigma * 1j * u + d) * T - 2 * np.log((1 - g * np.
            exp(d * T)) / (1 - g))))
    D = ((b - rho * sigma * 1j * u + d) / sigma**2) * ((1 - np.exp(d * T)) / (1 - g *
        np.exp(d * T)))
    return np.exp(C + D * v0 + 1j * u * np.log(S))

def heston_price(S, K, T, r, v0, theta, kappa, sigma, rho, option_type='Call'):
    def integr(u):
        return np.real(np.exp(-1j * u * np.log(K)) * heston_char(u - 1j, S, K, T, r, v0
            , theta, kappa, sigma, rho, option_type) / (1j * u * S * np.exp(-r * T)))
    integral = quad(integr, 0, 100)[0]
    price = S * 0.5 - K * np.exp(-r * T) / np.pi * integral
    return price if option_type == "Call" else price - (S - K * np.exp(-r * T))

def heston_greeks(S, K, T, r, v0, theta, kappa, sigma, rho, option_type, eps=1e-4):
    base = heston_price(S, K, T, r, v0, theta, kappa, sigma, rho, option_type)
    price_plus = heston_price(S + eps, K, T, r, v0, theta, kappa, sigma, rho,
        option_type)
    price_minus = heston_price(S - eps, K, T, r, v0, theta, kappa, sigma, rho,
        option_type)
    delta = (price_plus - price_minus) / (2 * eps)
    gamma = (price_plus - 2 * base + price_minus) / (eps ** 2)
    vega = (heston_price(S, K, T, r, v0 + eps, theta, kappa, sigma, rho, option_type)
        -
        heston_price(S, K, T, r, v0 - eps, theta, kappa, sigma, rho, option_type))
        / (2 * eps)
    return delta, gamma, vega

def options_with_heston(df, v0=0.04, theta=0.04, kappa=1.5, sigma=0.3, rho=-0.7):
    df = df.copy()
    heston_prices = []
    deltas = []
    gammas = []
    vegas = []
    for _, row in df.iterrows():
        S = row['S (spot)']
        K = row['K (strike)']
        T = row['T (days)'] / 365
        option_type = row['Type']

```

```

        price = heston_price(S, K, T, r, v0, theta, kappa, sigma, rho, option_type)
        delta, gamma, vega = heston_greeks(S, K, T, r, v0, theta, kappa, sigma, rho,
            option_type)
        heston_prices.append(price)
        deltas.append(delta)
        gammas.append(gamma)
        vegas.append(vega)
    df['Price (Heston)'] = heston_prices
    df['Delta (Heston)'] = deltas
    df['Gamma (Heston)'] = gammas
    df['Vega (Heston)'] = vegas
    return df

options1 = options_with_heston(options1)
options1.to_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/options1.
    csv")
options2 = options_with_heston(options2)
options2.to_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/options2.
    csv")
options3 = options_with_heston(options3)
options3.to_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/options3.
    csv")

#SABR Optimization

def optimize_sabr(df):
    df['Gamma (SABR)'] = df['Gamma (SABR)'].clip(upper=1.0)
    df['Vega (SABR)'] = df['Vega (SABR)'].clip(lower=-20, upper=20)
    delta = df['Delta (SABR)'].values
    gamma = df['Gamma (SABR)'].values
    vega = df['Vega (SABR)'].values
    n = len(df)
    w0 = np.ones(n) / n
    def maximize_vega(w):
        return -np.dot(w, vega)
    constraints = [{'type': 'eq', 'fun': lambda w: np.dot(w, delta)}, {'type': 'ineq',
        'fun': lambda w: 0.3 - abs(np.dot(w, gamma))}, {'type': 'eq', 'fun': lambda w:
        np.sum(w) - 1.0}]
    bounds = [(-1, 1) for _ in range(n)]
    result = minimize(maximize_vega, w0, constraints=constraints, bounds=bounds,
        method='SLSQP')
    df['Weight (SABR)'] = result.x
    print (df[['Date', 'Type', 'S (spot)', 'K (strike)', 'T (days)', 'Delta (SABR)', '
        Gamma (SABR)', 'Vega (SABR)', 'Weight (SABR)']])
    print(f"Total Delta : {np.dot(result.x, delta):.4f}")
    print(f"Total Gamma : {np.dot(result.x, gamma):.4f}")
    print(f"Total Vega : {np.dot(result.x, vega):.4f}")
    print(f"Sum of weights: {np.sum(result.x):.4f}")
    return df

options1 = optimize_sabr(options1)
options1.to_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/options1.
    csv")
options2 = optimize_sabr(options2)
options2.to_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/options2.
    csv")
options3 = optimize_sabr(options3)
options3.to_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/options3.
    csv")

```

```

    csv")

#Heston Optimization
def optimize_heston(df):
    df['Gamma (Heston)'] = df['Gamma (Heston)'].clip(upper=1.0)
    df['Vega (Heston)'] = df['Vega (Heston)'].clip(lower=-20, upper=20)
    delta = df['Delta (Heston)'].values
    gamma = df['Gamma (Heston)'].values
    vega = df['Vega (Heston)'].values
    n = len(df)
    w0 = np.ones(n) / n
    def maximize_vega(w):
        return -np.dot(w, vega)
    constraints = [{ 'type': 'eq', 'fun': lambda w: np.dot(w, delta)}, { 'type': 'ineq',
        'fun': lambda w: 0.3 - abs(np.dot(w, gamma))}, { 'type': 'eq', 'fun': lambda w:
        np.sum(w) - 1.0}]
    bounds = [(-1, 1) for _ in range(n)]
    result = minimize(maximize_vega, w0, constraints=constraints, bounds=bounds,
        method='SLSQP')
    df['Weight (Heston)'] = result.x
    print (df[['Date', 'Type', 'S (spot)', 'K (strike)', 'T (days)', 'Delta (Heston)', '
        Gamma (Heston)', 'Vega (Heston)', 'Weight (Heston)']])
    print(f"Total Delta : {np.dot(result.x, delta):.4f}")
    print(f"Total Gamma : {np.dot(result.x, gamma):.4f}")
    print(f"Total Vega : {np.dot(result.x, vega):.4f}")
    print(f"Sum of weights: {np.sum(result.x):.4f}")
    return df

options1 = optimize_heston(options1)
options1.to_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/options1.
    csv")
options2 = optimize_heston(options2)
options2.to_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/options2.
    csv")
options3 = optimize_heston(options3)
options3.to_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/options3.
    csv")

#Backtesting

prices = pd.read_csv("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/
    wti_prices.csv", skiprows=2)
prices.columns = ['Date', 'Close']
prices['Date'] = pd.to_datetime(prices['Date'])
prices.set_index('Date', inplace=True)

start_dates = ["2020-03-02", "2021-01-04", "2021-07-01"]
portfolios = [options1, options2, options3]

for start_date_str, df in zip(start_dates, portfolios):
    start_date = pd.to_datetime(start_date_str)
    total_payoff_heston = 0
    total_payoff_sabr = 0
    vega_heston = 0
    vega_sabr = 0
    for maturity_days in [30, 90, 180]:
        maturity_date = start_date + pd.Timedelta(days=maturity_days)
        while True:

```

```

try:
    S_T = float(prices.loc[maturity_date]['Close'])
    break
except KeyError:
    maturity_date += pd.Timedelta(days=1)
    continue
sub_df = df[df['T (days)'] == maturity_days].copy()
sub_df['Payoff'] = np.where(sub_df['Type'] == 'Call', np.maximum(S_T - sub_df['K (strike)'], 0), np.maximum(sub_df['K (strike)'] - S_T, 0)) - sub_df['Price (BSM)']
sub_df['S_T'] = S_T
sub_df['Weighted Payoff (Heston)'] = sub_df['Weight (Heston)'] * sub_df['Payoff']
sub_df['Weighted Payoff (SABR)'] = sub_df['Weight (SABR)'] * sub_df['Payoff']
sub_df['Weighted Vega (Heston)'] = sub_df['Weight (Heston)'] * sub_df['Vega (Heston)']
sub_df['Weighted Vega (SABR)'] = sub_df['Weight (SABR)'] * sub_df['Vega (SABR)']
sub_df = sub_df[['Date', 'Type', 'T (days)', 'K (strike)', 'S_T', 'Price (BSM)', 'Payoff', 'Weighted Payoff (Heston)', 'Weighted Payoff (SABR)', 'Weighted Vega (Heston)', 'Weighted Vega (SABR)']]
total_payoff_heston += sub_df['Weighted Payoff (Heston)'].sum()
total_payoff_sabr += sub_df['Weighted Payoff (SABR)'].sum()
vega_heston += sub_df['Weighted Vega (Heston)'].sum()
vega_sabr += sub_df['Weighted Vega (SABR)'].sum()
pnl_per_vega_heston = total_payoff_heston / abs(vega_heston)
pnl_per_vega_sabr = total_payoff_sabr / abs(vega_sabr)
print(f"Portfolio starting on {start_date_str}:")
print(f" Total PnL (Heston) = {total_payoff_heston:.2f}")
print(f" Total PnL (SABR) = {total_payoff_sabr:.2f}")
print(f" PnL per unit of Vega (Heston) = {pnl_per_vega_heston:.2f}")
print(f" PnL per unit of Vega (SABR) = {pnl_per_vega_sabr:.2f}")

results = {
    'Start Date': ['2020-03-02', '2021-01-04', '2021-07-01'],
    'Heston': [16.46, 7.80, -3.87],
    'SABR': [-21.94, 16.54, -1.56]
}
df = pd.DataFrame(results)

results2 = {
    'Start Date': ['2020-03-02', '2021-01-04', '2021-07-01'],
    'Heston': [0.12, 0.06, -0.03],
    'SABR': [-0.44, 0.32, -0.02]
}
df2 = pd.DataFrame(results2)

bar_width = 0.35
x = range(len(df))
y_min = min(df['Heston'].min(), df['SABR'].min()) - 5
y_max = max(df['Heston'].max(), df['SABR'].max()) + 5
heston_bars = plt.bar(x, df['Heston'], width=bar_width, label='Heston')
sabr_bars = plt.bar([p + bar_width for p in x], df['SABR'], width=bar_width, label='SABR')
for bar in heston_bars:
    yval = bar.get_height()
    offset = 1.2 if yval >= 0 else -2
    plt.text(bar.get_x() + bar.get_width()/2, yval + offset, f'{yval:.2f}',
             ha='center', va='bottom' if yval >= 0 else 'top')
for bar in sabr_bars:

```

```

    yval = bar.get_height()
    offset = 1.2 if yval >= 0 else -2
    plt.text(bar.get_x() + bar.get_width()/2, yval + offset, f'{yval:.2f}',
             ha='center', va='bottom' if yval >= 0 else 'top')

plt.xlabel('Portfolio Start Date')
plt.ylabel('Total PnL')
plt.xticks([p + bar_width / 2 for p in x], df['Start Date'])
plt.axhline(0, color='black', linewidth=0.8)
plt.ylim(y_min, y_max)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/pnl.png")
plt.show()

bar_width = 0.35
x = range(len(df2))
y_min = -0.5
y_max = 0.5
hestonBars = plt.bar(x, df2['Heston'], width=bar_width, label='Heston')
sabrBars = plt.bar([p + bar_width for p in x], df2['SABR'], width=bar_width, label='
    SABR')
for bar in hestonBars:
    yval = bar.get_height()
    offset = 0.015 if yval >= 0 else -0.025
    plt.text(bar.get_x() + bar.get_width()/2, yval + offset, f'{yval:.2f}',
             ha='center', va='bottom' if yval >= 0 else 'top', fontsize=10)

for bar in sabrBars:
    yval = bar.get_height()
    offset = 0.015 if yval >= 0 else -0.025
    plt.text(bar.get_x() + bar.get_width()/2, yval + offset, f'{yval:.2f}',
             ha='center', va='bottom' if yval >= 0 else 'top', fontsize=10)
plt.xlabel('Portfolio Start Date', fontsize=12)
plt.ylabel('PnL per Vega', fontsize=12)
plt.xticks([p + bar_width / 2 for p in x], df2['Start Date'], fontsize=10)
plt.yticks(fontsize=10)
plt.axhline(0, color='black', linewidth=0.8)
plt.ylim(y_min, y_max)
plt.legend()
plt.grid(True, axis='y')
plt.tight_layout()
plt.savefig("C:/Users/leama/OneDrive - Universit Cte d'Azur/Mmoire/Code/pnlvega.png")
plt.show()

```